



Onboarding Databases to DSF Hub Reference Guide

2 June 2026

Contents

Amazon RDS for PostgreSQL via CloudWatch Onboarding Steps 3

Amazon RDS for PostgreSQL via CloudWatch Onboarding Steps

This topic reviews the steps required to audit activity on Amazon RDS for PostgreSQL via CloudWatch databases and send that information to DSF Hub for auditing, analysis, detecting and preventing security events.

Getting Started

This page contains the information necessary to onboard this data source to Data Security Fabric (DSF) Hub. The following main topics are included:

- An overview of the available scope for the data source, given its native audit data.
- A complete list of prerequisites and permissions that are required for onboarding data sources to DSF Hub.
- Instructions on how to enable audit on the data source and collect data using DSF Hub.
- Initial troubleshooting steps and technical support information.

Audit Data Scope

The scope of audit data collection varies between data sources. Prior to enabling auditing, read through the auditing scope for this data source to identify any potential benefits or constraints. For more information on data collections, see [Main Collections in sonargd Database](#).

- This source collects audit events in the **instance**, **exception**, and **session** collections. Optionally, events can also be collected in the **slow_query** collection.
- When the source generates multiple log lines for a single action, it uses metrics to aggregate them into a single document and stores it in the instance, exception, or session collection based on the event type.
- An exception occurs when a syntactically correct query fails during execution. In this case, two log events may be generated with different classifications: one successful `LOG: 00000: AUDIT:` event and one failed `ERROR/DETAIL/STATEMENT` event. Because they differ in classification and no transaction or query ID is available to correlate them, they cannot be merged and are stored as two separate documents.

Onboarding Steps

To ensure a smooth and successful deployment, it is necessary to complete each of these onboarding steps. Please click on each of the steps below to display the content.

✔STEP 1: Gathering Permissions & Information

Onboarding a data source requires preparation, such as gathering permissions and collecting relevant information for your deployment. Assistance may be necessary from a database administrator, network administrator, and an IT administrator to successfully begin monitoring your data source.

Please ensure all of the following items are properly configured or available for use. The information and permissions gathered in this step are required for the remaining onboarding steps.

Cloud Account Prerequisites

Below are the requirements for an Agentless Gateway to communicate with data sources and their related endpoints. Please ensure these are completed by a Cloud Administrator.

Permissions to Create or Modify AWS Resources

For AWS RDS for PostgreSQL Instance:

- `rds:CreateDBParameterGroup` and `rds:ModifyDBParameterGroup` - used to create a parameter group and add the necessary parameters.
- `rds:ModifyDBInstance` - used to attach the parameter group to the Amazon RDS For PostgreSQL instance.

For AWS RDS for PostgreSQL Cluster:

- `rds:CreateDBClusterParameterGroup` and `rds:ModifyDBClusterParameterGroup` - used to create a cluster parameter group and set the necessary parameters.
- `rds:ModifyDBInstance` - used to attach the Cluster Parameter Group to the Amazon RDS For PostgreSQL Multi AZ Cluster.

AWS Cloud Account authentication mechanisms

DSF supports the following authentication mechanisms for accessing AWS resources:

- Key pair: for more information on creating a key pair, see [Manage access keys for IAM users](#).
- IAM roles: for more information on creating an IAM role, see [IAM roles for Amazon EC2](#).
- Profile
- Default

The authentication mechanism assigned to the Agentless Gateway should be granted the following privileges:

- To use DSF's Discovery Feature:

```
logs:DescribeLogGroups
rds:DescribeDBInstances
```

```
For Cluster Deployments use also:
rds:DescribeDBClusters
```

- To access the log group and retrieve the audit logs:

```
logs:DescribeLogGroups
logs:DescribeLogStreams
logs:FilterLogEvents
logs:GetLogEvents
```

- To use DSF's audit policy management feature:

```
rds:DescribeDBInstances
rds:DescribeDBParameterGroups
rds:DescribeOptionGroups
rds:DescribeDBSecurityGroups
ec2:DescribeSecurityGroups
```

Database Prerequisites

Below are the requirements for granting database permissions for audit log retrieval. Please ensure these are completed by a Database Administrator.

- Verify you are able to connect to the Amazon RDS for PostgreSQL server using the "master" user to create the PGAudit Extension.
- A user that is assigned the "rds_superuser" role to create and manage audit policies. (Optional)

Creating the audit management user (Optional)

If using the Audit Policy Management Feature (see Step 4 "Managing Audit Policies"), you need to create a dedicated audit management user. This user will connect to the database and run queries to create and modify audit policies.

To create the audit management user, connect to the RDS for PostgreSQL database. As the "master" user, create an auditmgr user and grant the rds_super_user role.

```
CREATE USER auditmgr with password '<password>';
GRANT rds_superuser TO auditmgr;
ALTER USER auditmgr WITH CREATEROLE;
```

✓STEP 2: Enabling Audit on the Data Source

Once the required permissions and information have been obtained, please complete the following steps to enable audit on the data source.

Enabling audit on RDS for PostgreSQL

Please **choose only one** of the following methods to enable audit logging:

- Enabling Audit via AWS console
- Enabling Audit via AWS CLI

Note:

Audit retrieval from Multi-AZ DB **Cluster** deployments are supported in DSF version 4.19 +. For more

information, see [Configuring and managing a Multi-AZ deployment](#).

Enabling audit via AWS console

This section provides instructions for creating the parameter group which is assigned to the Amazon RDS For PostgreSQL server. If an existing parameter group is being used, skip to the next section (Modify the DB Parameter Group).

1. Create or Update a Parameter Group

Note:

The parameter group must be in the same region as the Amazon RDS For PostgreSQL server.

To create a parameter group, navigate to the [Amazon RDS console](#) and select "Parameter Groups" on the navigation pane.

- a. Choose Create parameter group. The Create parameter group window appears.
- b. In the Create parameter group window set the field values as indicated below.
 - a. Group Name - give an identifier for the parameter group.
 - b. Description - short description for the parameter group.
 - c. Engine type- PostgreSQL.
 - d. Parameter Group Family - select the version of your current data source.
 - e. Type - choose according to your RDS PostgreSQL deployment type:
 - a. For RDS PostgreSQL Instance and Multi AZ **Instance** choose: DB Parameter Group.
 - b. For RDS PostgreSQL Multi AZ **Cluster** choose: DB Cluster Parameter Group.
 - f. Choose Create.

For more information, please refer to the AWS documentation: [Working with DB parameter groups for Amazon RDS DB instances](#) and [Working with DB cluster parameter groups for Multi-AZ DB clusters](#).

2. Modify the DB Parameter Group

- a. Navigate to your DB Parameter Group using the group name.
- b. On the top right, select "Edit parameters" button.

c. Change the following parameters and assign them the exact values listed below:

Parameter	Value	Notes
shared_preload_libraries	pgaudit	Add to existing values
log_connections	receipt,authentication,authorization	for PostgreSQL < 18: use 1 instead
log_disconnections	1	
log_error_verbosity	verbose	
pgaudit.log	all	Setting the pgaudit.log parameter to all will log all events. To set up a more granular Audit Policy, see Step 4 "Managing Audit Policies".

d. (Optional) Turn on Slow Query Monitoring:

Note:

- Turning on Slow Query Monitoring will not interfere with standard audit collection.
- Queries that exceed the minimum duration threshold will be mapped to a separate collection called sonargd.slow_query.
- This step is **optional** and required only if you choose to audit slow query logs.
- The following setting should be done **in addition** to all previous steps.

Slow query logs audit SQL statements that run longer than a specified duration. This duration is set using the parameter "log_min_duration_statement" in the DB parameter group.

To turn on slow query monitoring modify the DB Parameter Group and change the following parameters where <value> is your desired threshold in milliseconds:

Parameter	Value	Notes
log_statement	None	Must be set to none for slow query logs to ingest correctly. Any other value may cause slow query logs to be ignored.
log_duration	0	Enables duration logging. Setting to 0 logs the duration of every completed statement. Can increase log volume significantly.
log_min_duration_statement	<value>(in milliseconds)	Threshold for slow query logging. Only queries exceeding this duration will be logged to sonargd.slow_query. Replace <value> with your chosen threshold.

- e. Select the "Save changes" button on the top right.

3. Attach the DB Parameter Group to the PostgreSQL server

- a. Navigate to the [Amazon RDS console](#) through the AWS console.
- b. In the navigation pane, choose Databases.
- c. Select your Amazon RDS For PostgreSQL server.
- d. Choose Modify.
- e. Under the Additional Configuration select your DB parameter group from the dropdown.
- f. Choose Continue.
- g. Select when to apply the modifications.
- h. Select Modify on the summary page.

4. Redirect Audit Logs to CloudWatch Log Group

-
- a. Navigate to the [Amazon RDS console](#) through the AWS console.
 - b. In the navigation pane, choose Databases.
 - c. Select your Amazon RDS For PostgreSQL server.
 - d. Choose Modify.
 - e. In the Log exports section, choose PostgreSQL log.
 - f. Choose Continue.
 - g. Select when to apply the modifications.
 - h. Select Modify on the summary page.
5. Reboot the RDS PostgreSQL server to apply the new changes.
 6. **To finish enabling audit for your RDS for PostgreSQL server** continue with the **Creating the PGAudit Extension** section.

Enabling Audit via AWS CLI

For Amazon RDS For PostgreSQL Instance and Multi AZ Instance Deployments

1. Create or Update a Parameter Group

To create a DB parameter group, use the AWS CLI [create-db-parameter-group](#) command and include the following required parameters:

- `--db-parameter-group-name`
- `--db-parameter-group-family`
- `--description`

The following example creates a DB parameter group named "mydbparametergroup" with "Postgres14" as a parameter group family, and a description of "My new parameter group.". Please change these values for your DB parameter group.

On Linux, macOS, or Unix:

```
aws rds create-db-parameter-group \  
  --db-parameter-group-name mydbparametergroup \  
  --db-parameter-group-family Postgres14 \  
  --description "My new parameter group"
```

2. Modify the DB Parameter Group

To modify a DB parameter group, use the AWS CLI `modify-db-parameter-group` command with the following required parameters:

- `--db-parameter-group-name`
- `--parameters`

The following example modifies all of the required parameters in the DB parameter group named "mydbparametergroup".

For Linux, macOS, or Unix:

PostgreSQL < 18:

```
aws rds modify-db-parameter-group \
  --db-parameter-group-name mydbparametergroup \
  --parameters \
    "ParameterName=shared_preload_libraries,ParameterValue=pgaudit,ApplyMethod=immediate" \
    "ParameterName=pgaudit.log,ParameterValue=all,ApplyMethod=immediate" \
    "ParameterName=log_connections,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=log_disconnections,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=log_error_verbosity,ParameterValue=verbose,ApplyMethod=immediate"
```

PostgreSQL ≥ 18:

```
aws rds modify-db-parameter-group \
  --db-parameter-group-name mydbparametergroup \
  --parameters \
    "ParameterName=shared_preload_libraries,ParameterValue=pgaudit,ApplyMethod=immediate" \
    "ParameterName=pgaudit.log,ParameterValue=all,ApplyMethod=immediate" \
    "ParameterName=log_connections,ParameterValue='receipt, authentication, authentication_failed'", \
    "ParameterName=log_disconnections,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=log_error_verbosity,ParameterValue=verbose,ApplyMethod=immediate"
```

****** Setting the `pgaudit.log` parameter to `all` will log all events. To set up a more granular Audit Policy, see Step 4 "Managing Audit Policies".

(Optional) Turn on Slow Query Monitoring:

Note:

- Turning on Slow Query Monitoring will not interfere with standard audit collection.
- Queries that exceed the minimum duration threshold will be mapped to a separate collection called `sonargd.slow_query`.
- Make sure the **log_statement** parameter is set to **None**. Any modifications may cause slow query logs to not be ingested.
- Make sure the **log_duration** parameter is set to **0**. Enabling this parameter can cause the log file to fill up rapidly, potentially leading to delays in log ingestion.
- This step is **optional** and required only if you choose to audit slow query logs.
- The following setting should be done IN ADDITION to all the previous steps.

Slow query logs audit SQL statements that run longer than a specified duration. This duration is set using the parameter `log_min_duration_statement` in the DB Parameter Group.

To turn on slow query monitoring, modify the DB Parameter Group and change the following parameter: `log_min_duration_statement`. The value should be your desired threshold in milliseconds.

The following example modifies the `log_min_duration_statement` Parameter and sets the threshold to 5000 milliseconds, in the DB Parameter Group named `mydbclusterparametergroup`.

```
aws rds modify-db-parameter-group \
  --db-parameter-group-name mydbparametergroup \
  --parameters \
    "ParameterName=log_min_duration_statement,ParameterValue=5000,ApplyMethod=immediate" \
    "ParameterName=log_statement,ParameterValue=none,ApplyMethod=immediate" \
    "ParameterName=log_duration,ParameterValue=0,ApplyMethod=immediate"
```

3. Attach the DB Parameter Group to the PostgreSQL server and redirect the Audit Logs to CloudWatch Log Group

Use the `modify-db-instance` command with the following required parameters:

- `--db-instance-identifier`
- `--db-parameter-group-name`
- `--cloudwatch-logs-export-configuration`

```
aws rds modify-db-instance \
```

```
--db-instance-identifier <db-instance-name> \
--db-parameter-group-name <parameter-group-name> \
--cloudwatch-logs-export-configuration '{"EnableLogTypes":["postgresql"]}'
--apply-immediately
```

4. Reboot the RDS for PostgreSQL server to apply the new changes.
5. To finish enabling audit for your RDS for PostgreSQL server, move on to the below **Creating the PGAudit Extension** section.

For Amazon RDS For PostgreSQL Multi AZ Cluster Deployments

1. Create or Update a Parameter Group

Use the AWS CLI `create-db-cluster-parameter-group` command and include the following required parameters.

The following example creates a DB cluster parameter group named "mydbclusterparametergroup" for PostgreSQL version 14, with a description of "My new cluster parameter group."

For Linux, macOS, or Unix:

```
aws rds create-db-cluster-parameter-group \
--db-cluster-parameter-group-name mydbclusterparametergroup \
--db-parameter-group-family Postgres14 \
--description "My new cluster parameter group"
```

For more information, please refer to the AWS document: [Working with DB parameter groups](#).

2. Modify the DB Parameter Group

To modify a DB parameter group, use the AWS CLI `modify-db-parameter-group` command with the following required parameters:

- `--db-parameter-group-name`
- `--parameters`

The following example modifies the required parameters in the DB parameter group named "mydbparametergroup".

For Linux, macOS, or Unix:

PostgreSQL < 18:

```
aws rds modify-db-cluster-parameter-group \
--db-parameter-group-name mydbclusterparametergroup \
--parameters \
```

```
"ParameterName=shared_preload_libraries,ParameterValue=pgaudit,ApplyMethod=pending-reboot" \
"ParameterName=pgaudit.log,ParameterValue=all,ApplyMethod=immediate" \
"ParameterName=log_connections,ParameterValue=1,ApplyMethod=immediate" \
"ParameterName=log_disconnections,ParameterValue=1,ApplyMethod=immediate" \
"ParameterName=log_error_verbosity,ParameterValue=verbose,ApplyMethod=immediate"
```

PostgreSQL ≥ 18:

```
aws rds modify-db-cluster-parameter-group \
--db-parameter-group-name mydclusterbparametergroup \
--parameters \
"ParameterName=shared_preload_libraries,ParameterValue=pgaudit,ApplyMethod=pending-reboot" \
"ParameterName=pgaudit.log,ParameterValue=all,ApplyMethod=immediate" \
"ParameterName=log_connections,ParameterValue='receipt, authentication, authorization',ApplyMethod=immediate" \
"ParameterName=log_disconnections,ParameterValue=1,ApplyMethod=immediate" \
"ParameterName=log_error_verbosity,ParameterValue=verbose,ApplyMethod=immediate"
```

** Setting the **pgaudit.log** parameter to **all** will log all events. To set up a more granular Audit Policy, see Step 4 "Managing Audit Policies".

(Optional) Turn on Slow Query Monitoring:

Note:

- Turning on Slow Query Monitoring will not interfere with standard audit collection.
- Queries that exceed the minimum duration threshold will be mapped to a separate collection called `sonargd.slow_query`.
- This step is **optional** and required only if you choose to audit slow query logs.
- Make sure the **log_statement** parameter is set to **None**. Any modifications may cause slow query logs to not be ingested.
- Make sure the **log_duration** parameter is set to **0**. Enabling this parameter can cause the log file to fill up rapidly, potentially leading to delays in log ingestion
- The following setting should be done **in addition** to all the previous steps.

Slow query logs audit SQL statements that run longer than a specified duration. This duration is set using the parameter `"log_min_duration_statement"` in the DB Parameter Group.

To turn on slow query monitoring modify the DB Parameter Group and change the following parameter:

`log_min_duration_statement`. The value should be your desired threshold in milliseconds.

The following example modifies the `"log_min_duration_statement"` Parameter and sets the threshold to 5000 milliseconds, in the DB Cluster Parameter Group named `mydbclusterparametergroup`.

```
aws rds modify-db-cluster-parameter-group \
    --db-cluster-parameter-group-name mydbclusterparametergroup \
    --parameters \
        "ParameterName=log_min_duration_statement,ParameterValue=5000,ApplyMethod=pending-reboot" \
        "ParameterName=log_statement,ParameterValue=none,ApplyMethod=immediate" \
        "ParameterName=log_duration,ParameterValue=0,ApplyMethod=immediate"
```

3. Attach the DB Parameter Group to the PostgreSQL server and redirect the Audit Logs to CloudWatch Log Group

For **Cluster** deployments use the `modify-db-cluster` command with the following required parameters:

```
aws rds modify-db-cluster \
    --db-cluster-identifier <db-clus-name> \
    --db-parameter-group-name <parameter-group-name> \
    --cloudwatch-logs-export-configuration '{"EnableLogTypes":["postgresql"]}' --apply-immedi
```

4. Reboot the RDS for PostgreSQL server to apply the new changes.
5. To finish enabling audit for your RDS for PostgreSQL server, move on to the below **Creating the PGAudit Extension** section.

Creating the PGAudit Extension

1. Login to your RDS for PostgreSQL server as "master" user with a PostgreSQL compatible client and confirm the `pgaudit` plugin has been installed by running the following command:

```
SHOW shared_preload_libraries;
```

The output should be similar to:

```
Shared_preload_libraries
-----
Pgaudit
```

2. Create the extension by running the following command:

```
CREATE EXTENSION pgaudit;
```

3. Verify the extension has been successfully installed with the following command:

```
postgres=> \dx
List of installed extensions
Name | Version | Schema | Description
-----+-----+-----+-----
pgaudit | 16.0 | public | provides auditing functionality
plpgsql | 1.0 | pg_catalog | PL/pgSQL procedural language
(2 rows)
```

Note:

DSF requires the installation of the `pgaudit` extension on **all** databases from which you wish to receive audit logs. While auditing can still function without the extension, some information may be incomplete or missing.

You should now see audit logs in the `pgaudit` format being written to a CloudWatch Log Group with a path typically structured like this:

- For RDS for PostgreSQL Instance and Multi AZ Instance deployments:

```
/aws/rds/instance/<your-db-instance-identifier>/postgresql
```

- For RDS for PostgreSQL Multi AZ Cluster deployments:

```
/aws/rds/cluster/<your-db-cluster-identifier>/postgresql
```

(Optional) Space Management of CloudWatch Log Group Log Events

CloudWatch logs do not expire or get deleted; however, a log retention period may be customized to delete log events after a certain period of time. This does not delete log streams or log groups. Follow the steps below to set a retention period for the CloudWatch log group using either the console or CLI:

To set via the CloudWatch console:

1. Navigate to the home page of the log group.
2. Click on "Actions" then "Edit retention setting".
3. Choose a retention period setting.
4. Click "Save".

To set via the AWS CLI:

```
aws logs put-retention-policy --log-group-name <log-group-name> --retention-in-days <value>
```

Note that only specific values are permitted for the `--retention-in-days` parameter, see [PutRetentionPolicy Request Syntax](#).

✓STEP 3: Collecting Audit Data

After completing the prerequisites and enabling audit, the data source is ready to be onboarded onto DSF. This can be accomplished using **ANY ONE** of the methods listed below:

- Importing Assets via Unified Settings Console (USC)
- Importing Assets via an Asset Template Spreadsheet
- Importing Assets via DSF Open APIs

Please use the Asset Specifications documents below as a guide to fill in the field values for this data source and if applicable, its related assets.

[AWS Asset Specifications](#)

[AWS RDS PostgreSQL Asset Specifications](#)

[AWS Log Group Asset Specifications](#)

Note:

When onboarding an Amazon data source that exports audit data via **Amazon CloudWatch**, three assets are required:

- An **AWS Cloud Account** asset
- A **Database or Instance** asset
- An **AWS Log Group** asset

Define the relationship between these assets using the `parent_asset_id` field (or **Parent Asset Name** when using USC). The asset hierarchy should be as follows:

- The **AWS Cloud Account** is the parent of the **Database or Instance** asset.
- The **Database or Instance** asset is the parent of the **AWS Log Group** asset.

Refer to the asset template spreadsheet for a visual demonstration of the asset structure and relationship. For download instructions, see [Downloading Asset Templates](#).

Importing Assets via Unified Settings Console (USC)

USC allows users to configure a full audit flow, including importing new data assets. To add a new data source asset in USC, please follow these steps:

1. From the DSF homepage of your DSF Hub, under **Apps**, click **Unified Settings Console**. In the Appliances pane, select **DSF Hub**.

-
2. Click the Data Sources tab to view the data sources of the DSF Hub. Click **Add** to open the Add Data Source form.
 3. In the Data Source Type dropdown menu, select a data source.
 4. Specific data source configuration sections will display: Details, Connections, and Monitoring. Configure the mandatory configuration fields under Details and any optional configuration fields displayed under Advanced.
 5. If required for the data source, under Connection, select an authentication method (Auth Mechanism) from the dropdown menu. The mandatory fields for the selected Auth Mechanism are displayed; to see optional configuration fields, click Advanced.
 6. Under Monitoring, select the Agentless Gateway that will own the data source in the Gateway field, then click **Save**.
 7. Locate the asset you want to connect to the Agentless Gateway. Click on the kebab menu to the right of your asset, then **Enable Audit Collection** to start collecting audit data.

For more information on adding, viewing and editing assets in USC, see the [Adding Assets via Unified Settings Console \(USC\)](#) documentation.

Discover Assets using Cloud Accounts in USC

The Cloud Accounts tab allows you to run the Discovery playbook at any time to automatically discover and onboard new assets. Follow the instructions below to add a new cloud account and run Discovery via USC.

To add a new cloud account:

1. From the DSF homepage of your DSF Hub, under **Apps**, click **Unified Settings Console**. In the Appliances pane, select **DSF Hub**.
2. Click the Cloud Accounts tab to view the cloud accounts of the DSF Hub. Click **Add** to open the Add Cloud Account form.
3. In the Cloud Account Type dropdown menu, select Amazon, Azure, or Google Cloud.
4. Specific configuration sections will display: Details, Connections, and Monitoring. Configure the mandatory configuration fields under Details and any optional configuration fields displayed under Advanced.
5. Under Connection, select the authentication mechanism that the Agentless Gateway will use to connect to the cloud account.
 - For Amazon: in the Auth Mechanism dropdown menu, select Profile, Key, IAM Role or Default.
 - For Azure: in the Auth Mechanism dropdown menu, select Client Secret, Auth File, or Managed Identity.
 - For Google Cloud: in the Auth Mechanism dropdown menu, select Default or Service Account.
6. Depending on the selected cloud account type and authentication mechanism, different fields will be displayed under the Connection section. Fill them out accordingly.
7. Under Monitoring, select the Agentless Gateway that will own the cloud account in the Gateway field, then click **Save**.

To run Discovery in USC:

1. Locate the desired cloud account asset in USC as described above.
 2. Click on the kebab menu icon to the right of the cloud account on which you want to run the Discovery feature.
-

3. Click **Start Discovery**. Discovery will start running in the background.
4. After Discovery has run successfully, locate the asset you want to connect to the Agentless Gateway. Click on the kebab menu to the right of your asset, then **Enable Audit Collection** to start collecting audit data.

For more information on importing data source assets using cloud accounts in USC, see the [Discovering Data Sources in DSF Hub Cloud Accounts using USC](#) documentation.

Importing Assets via an Asset Template Spreadsheet

Complete these steps to import data source assets using asset template spreadsheets:

1. From the DSF Portal of your DSF Hub, under **Apps**, click **Sync Spreadsheet**. In the overlay, click **Import Assets**.
2. On the Import Assets page, go to the **Assets Templates** dropdown menu. Select the template of the cloud account you want to import and click **Download**.
3. Use the Asset Specification documentation as a guide for filling in the necessary field values.
4. On the Import Assets page, go to the section named **Upload 'Assets and Connections to Import' spreadsheet**. Navigate to the spreadsheet that you filled out and click **Open**.
5. To complete the process of onboarding the asset with the asset template spreadsheet, click **'Validate All'**.
6. Check for any errors or warnings, make any desired changes and then click **Run 'Import Assets'**.
7. Click on the **Assets Dashboard** button to navigate to the Assets Dashboard page, then locate the data source asset that was imported.
8. Click the kebab menu to the right of the asset, then click **Management > Audit Actions > Connect Gateway** to start collecting audit data.

For more information, please see the [Adding Assets via the Import Assets Page](#) documentation.

Note:

Use the following asset templates when importing via spreadsheet:

- Asset template name for Amazon RDS PostgreSQL:
 - For Amazon RDS PostgreSQL instances (including Multi AZ instances): **AMAZON_RDS_POSTGRESQL_template.xlsx**
 - For Amazon RDS PostgreSQL clusters (including Multi AZ clusters): **AMAZON_RDS_POSTGRESQL_CLUSTER_template.xlsx**
- Asset template name for AWS account; choose ONE of the 4 templates listed below based on the type of authentication mechanism:
 - **AMAZON_EC2_DEFAULT_ROLE_template.xlsx**
 - **AMAZON_IAM_ROLE_template.xlsx**
 - **AMAZON_KEY_PAIR_template.xlsx**
 - **AMAZON_PROFILE_template.xlsx**
- When running connect gateway, you are given two options for **audit_type**. Please select the audit_type most suitable for your needs:
 - Standard audit collection: **"Log Group"**
 - Standard audit monitoring with aggregated queries: **"Aggregated"**

Discover Assets using Cloud Accounts in the Assets Dashboard

Note:

This section can be skipped if the data source asset(s) have already been onboarded using spreadsheets in the previous section.

After configuring and onboarding a cloud account asset with the necessary permissions, the Asset Discovery feature can find and onboard related data source assets with just a few clicks.

1. From the DSF homepage of your DSF Hub, under **Apps**, click **Sync Spreadsheet**. In the overlay, click **Import Assets**.
2. On the Import Assets page, go to the **Assets Templates** dropdown menu. Select the template of the cloud account you want to import and click **Download**.
3. Use the Asset Specifications documentation as a guide for completing the asset template spreadsheet for the cloud account asset.
4. On the Import Assets page, go to the section named **Upload 'Assets and Connections to Import' spreadsheet**. Navigate to the spreadsheet that you filled out and click **Open**.
5. To complete the process of onboarding the asset with the asset template spreadsheet, click **Validate All**.
6. Check for any errors or warnings, make any desired changes and then click **Run 'Import Assets'**.
7. Click on the **Assets Dashboard** button to navigate to the Assets Dashboard page, then locate the cloud account asset that was imported.
8. Click the kebab menu to the right of the asset, then click **Discovery**. Choose the data source type to be discovered to start the Discovery playbook.
9. After the Discovery playbook has run successfully, locate the desired data source asset.
10. Click the kebab menu to the right of the asset, then click **Management > Audit Actions > Connect Gateway** to start collecting audit data.

For more information on discovering assets, please see the [Adding Assets via Asset Discovery](#) documentation.

Importing Assets via DSF Open APIs

Data Security Fabric (DSF) Open APIs provide functions for onboarding and managing assets (log aggregators, cloud accounts, data sources, secret managers and other assets) via a RESTful API. For more information on the supported assets and how to onboard them, please see [Using DSF Open APIs](#).

✓STEP 4: Managing Audit Policies

Audit Policies are created to specify which actions will be collected from the database. This can be done in two ways:

- **Using the Audit Policy Management Feature**
(or)
- **Creating an Audit Policy manually**

Using the Audit Policy Management Feature

DSF provides automated solutions for managing and monitoring audit policies. Audit policies can be managed from the Asset Dashboard (OR) USC.

Managing Audit Policies with Asset Dashboard

To apply an audit policy via Asset Dashboard:

1. In the DSF Portal, under **Dashboards**, click **Asset Dashboard**.
2. In the top left-hand side of the **DSF DataVis** window, click **Dashboard**, and then click the **Asset Details** link. The **Asset Details** dashboard opens.
3. In the **Asset Details** dashboard in DSF DataVis, go to the **Assets** panel and locate the asset to which you want to apply the audit policy.
4. Click the three vertical dots at the end of the row for the asset, in the **Actions** column.
5. In the menu that opens, select **Management > Audit Actions > Turn on Audit Policy**. The **Turn On Audit Policy** window opens.
6. In the **Turn On Audit Policy** window, under **Audit Policy**, select an audit policy by type or by name.
7. Click **Submit**. The audit policy is turned on, that is, applied to the asset.
8. If you want, click **Refresh** to view the asset's audit policy details (e.g. policy_template_name) in the relevant row in the **Assets** panel.

For detailed instructions and supported database versions, see [Managing Audit Policies](#) in the user guide.

Managing Audit Policies with Unified Settings Console

To create and apply a new DSF Hub audit policy via USC:

1. From the main page of **DSF Portal**, under **Apps**, click **Unified Settings Console**.
2. In the **Appliances** pane, select **DSF Hub**.
3. Click the **Audit Policies** tab. The audit policies of the DSF Hub appliance are displayed.
4. Click **Add**.
5. In the **Configuration** tab, in the **Policy Name** field, type a name for the policy.
6. In the **Configuration** tab, select a policy type. Options available in this menu include **Native audit policy**, **Gateway audit policy** and **Combined audit policy**.
7. Select match criteria. Available options depend on audit type selected: For more information about these criteria, see [Supported Criteria for Audit Policies](#).
8. If you selected the **Exclude List** type, add at least one of the following sets:
 - DB User Name
 - Analyzed Client IP
 - Source Program

You can add multiple sets of **Exclude List** criteria via the **+ Add** button.
9. If you selected the **Action Groups** policy type, add one or more action groups. For more information, see [Viewing Lists of Actions for Action Groups](#).
10. Click the **Applied to** tab and select the data source assets to which the policy will be applied.

11. Turn **Audit collection** on or off. This option provides a convenient method to enable audit collection directly from the audit policy.
12. Click **Save**. DSF Hub begins updating the audit policy; this process may take a few minutes to complete.

For detailed instructions, see [Managing Audit Policies via USC](#).

To use this, it is necessary to create a dedicated audit management user. This user will connect to the database and run queries to create and modify audit policies.

Creating an Audit Policy Manually

Alternatively, it is possible to manually create and manage audit policies on the data source itself.

The value of **pgaudit.log** in the database's parameter group defines the default setting of what gets audited across the instance. To override the default setting defined in the parameter group, we can set the pgaudit.log value at the user, database, or table level.

For example, to disable auditing for a particular user, run the following SQL command:

```
ALTER USER myuser SET pgaudit.log TO 'NONE';
```

Then, you can use the following query to verify audit settings per user:

```
SELECT pg_roles.rolname, pg_roles.rolconfig[1] FROM pg_roles WHERE rolconfig[1] like 'pgaudi
```

You should see output similar to the following

```
user_name | user_specific_settings
-----+-----
myuser    | {pgaudit.log=NONE}
(1 row)
```

The custom settings can be reverted with the following query:

```
ALTER USER myuser RESET pgaudit.log;
```

For more information, see [Using pgAudit to log database activity](#).

Troubleshooting

Should you encounter any unexpected issues or behaviors, you may check the status of the following services and associated log files to help pinpoint the root cause. If additional assistance is needed at any time, technical support staff is available to help users of all technical levels via <https://supportportal.thalesgroup.com/csm>.

On the Agentless Gateway host(s) please review the following services and log files depending on your selected audit type:

Audit Type	Gateway Service	Command for verifying the Gateway service	Log file
------------	-----------------	---	----------

Log Group	gateway-aws@postgresql.service	<code>systemctl status -i gateway-aws@postgresql.service</code>	<code>\$JSONAR_LOGDIR/gateway/cloud/aws/postgresql/sonargateway.log</code>
Aggregated	gateway-aws@postgresql-aggregated.service	<code>systemctl status -i gateway-aws@postgresql-aggregated.service</code>	<code>\$JSONAR_LOGDIR/gateway/cloud/aws/postgresql-aggregated/sonargateway.log</code>

For more information...

Need Help? For assistance with any DSF Hub or related products, please contact Online Technical Support via <https://supportportal.thalesgroup.com/csm>. A team of technical customer success representatives are ready to assist users of all skill levels.

Related Topics: Below are links with related onboarding information and procedures:

- [Using DSF Open APIs](#): A guide to onboarding and managing assets via DSF Open APIs.
- [Adding assets via Unified Settings Console \(USC\)](#): Instructions to add and edit data source assets via the USC.
- [Adding Assets via the Import Assets Page](#): Instructions to add data source assets via the Asset Dashboard.
- [Using the Data Security Fabric Portal](#): An overview of the Data Security Fabric (DSF) portal.
- [Data Security Coverage Tool](#): Systems and version compatibility with DSF Hub.

Document generated by Confluence on May 29, 2026 02:42

Atlassian